Cloud Computing / AWS

AWS Application Load Balancer Practical Report

Amazon Web Services

This practical documents the evidenced configuration of an internet-facing Application Load Balancer (ALB), an HTTP listener, and an EC2 instance target group. Its intended purpose is to provide one browser-facing endpoint that can route HTTP requests to more than one backend server and use health checks to assess target availability.

Evidence-based reconstruction of the supplied AWS console screenshots

November 3, 2026

Evidence Scope and Limitations

All 27 supplied screenshots, `LoadPractical/1.png` through `LoadPractical/27.png`, were reviewed as one chronological practical. They document VPC and security-group context, two EC2 launch forms, ALB and target-group configuration, the active ALB and its DNS name, and two browser responses. The report includes the screenshots that substantiate these steps; figure numbers are sequential and captions retain the source screenshot number.

Several details require qualification. Screenshots 18 and 19 show a health-check interval of `2` seconds, outside the displayed range of 5–300 seconds, but the later target-group review in screenshot 21 shows that the interval was changed to a valid `5` seconds. Screenshot 21 also explicitly shows `Targets (0)` even though screenshot 20 shows both instances selected in the registration form. No later target-registration confirmation or target-health view is present. Separately, the ALB security group has zero inbound rules in its creation form; later security-group edit forms show port-80 rules on the backend groups sourced from the ALB group, but the final saved ALB inbound rules are not shown. The active ALB and browser responses are directly evidenced, but these gaps prevent a complete audit of the final target registration and firewall state.

1 Introduction

This practical concerns a Layer 7 (application-layer) load-balancing design in which an AWS Application Load Balancer accepts HTTP requests and forwards them to EC2 instances registered in a target group. A load balancer gives clients a single entry point instead of requiring them to select a backend server themselves. It can improve availability by routing to healthy registered targets and makes it easier to add or remove backend capacity as an application changes.[2, 1]

The screenshots show two Amazon Linux 2023 EC2 instances configured to run Nginx and return different page text, an internet-facing IPv4 ALB named `LoadBalance`, and an HTTP:80 listener associated with an instance target group named `Target1`. The ALB is shown as active, and two browser screenshots display different backend responses at the ALB DNS name. The report also records evidence gaps around target registration and the final security-group state rather than assuming every intermediate form value was successfully saved.

2 Objectives

The evidenced and intended learning objectives are to:

- distinguish an Application Load Balancer from other Elastic Load Balancing types;
- understand how a listener, its forwarding action, a target group, and registered EC2 targets fit together;
- identify the VPC, subnets, and Availability Zones selected for an internet-facing IPv4 load balancer;
- configure an HTTP listener and an instance-type target group for HTTP traffic on port 80;
- understand target health checks and the need for valid health-check settings; and
- identify the security-group permissions required for client-to-ALB and ALB-to-target communication, and explain what the screenshots do and do not verify.

Testing whether one ALB endpoint can return content from both backend servers is supported by the two supplied browser screenshots. Strict alternation, equal traffic distribution, ongoing target health, and resilience to a target failure are not demonstrated.

3 Prerequisites and Foundational Concepts

3.1 Amazon EC2

Amazon Elastic Compute Cloud (EC2) provides virtual servers called instances. An instance can run a web server and return an application response. In this design, the EC2 instances are the backends. Their names, launch configuration, and Nginx user-data scripts are visible. The target-selection screen shows both instances running in `ap-south-1b`.[1]

3.2 Application Load Balancer

An ALB is an Elastic Load Balancing service that handles HTTP and HTTPS at the application layer. It is a client-facing endpoint and evaluates listener rules to decide where a request should go. This is distinct from a Network Load Balancer, which operates at the connection level and is intended for different traffic and performance requirements. Figure 11 confirms that the Application Load Balancer option, rather than the Network or Gateway Load Balancer option, was chosen.[2]

3.3 Target Groups and Registered Targets

A target group is a logical set of destinations plus routing and health-check settings. In this practical's creation form, the group is named `Target1` and its target type is `Instances`. That choice means EC2 instances are registered by instance identity. The target group is not itself a listener or an instance: it is the collection to which a listener can forward requests. Screenshot 20 shows two running instances selected for registration on port 80. However, the following review screen states `Targets (0)` and `No targets added`; no later target-registration list is shown. The browser tests later return both server pages through the ALB DNS name, which shows that both responses were served at test time, but does not fill in the missing registration record or target-health status.[5]

3.4 Listeners

A listener checks for client connections on a configured protocol and port. The earlier listener form uses HTTP on port 80 and selects "Forward to target groups" while its target-group dropdown is still blank. Screenshot 22 later shows `Target1` selected as the default forwarding destination, with weight 1 (100%) and stickiness off. Listeners and target groups are separate resources: the listener receives and routes; the target group identifies destinations. [4, 2]

3.5 Health Checks

A health check is a periodic request from the load balancer to each registered target. It tests whether the target responds as configured; it is not the same as a browser test. The screenshots show HTTP checks against path `/` on the target's traffic port, expected response code `200`, timeout 2 seconds, and thresholds of 5 successes and 2 failures. The interval is initially entered as 2 seconds in screenshots 18–19, then shown as 5 seconds on the review screen in screenshot 21. A target must be registered and reachable on the health-check port for these checks to determine its health; no target-health result is shown.[6]

3.6 Security Groups

Security groups are stateful virtual firewalls associated with AWS resources. They allow or deny network traffic through inbound and outbound rules. For an ALB design, client traffic must be allowed

into the ALB on its listener port; the ALB must be able to reach its targets; and target instances must allow the ALB's traffic, including health checks, on the relevant port. Restricting the target-side rule to the ALB security group is preferable to exposing backend HTTP broadly. Screenshots 23 and 24 show the latter rule being entered for the two backend groups; the Save rules buttons remain visible, so the captures do not prove that these edits were saved. The ALB security group form earlier showed zero inbound rules, and no later final ALB ingress-rule view is supplied. The subsequent successful browser responses show that the endpoint was reachable during testing, but do not establish the exact final ingress rule or its source range.[7, 3]

3.7 HTTP, TCP, and Ports

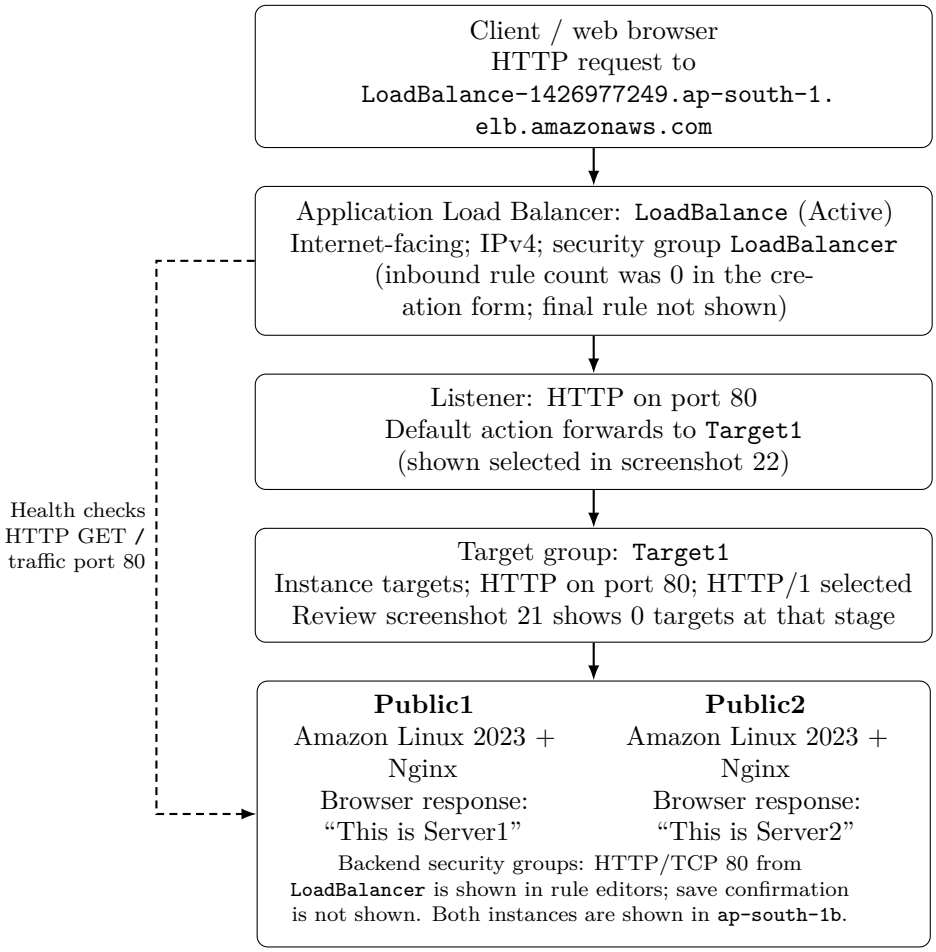
HTTP is the application protocol used by the browser, listener, target group, and health-check configuration shown here. HTTP communication normally uses TCP as its transport; port 80 is the conventional HTTP port and is the explicit port value in the listener and target-group forms. TCP is not interchangeable with HTTP: a port number identifies a transport endpoint, while HTTP defines the request/response semantics carried over that connection.

3.8 Availability Zones

An Availability Zone (AZ) is a distinct location within an AWS Region. A subnet belongs to one AZ. The screenshots show the load balancer mapped to subnets in `ap-south-1a` and `ap-south-1b`; selecting multiple AZs provides the network placement for the ALB across those zones. Screenshot 20 shows both backend instances in `ap-south-1b`; they are not evidenced as spread across both zones.[8, 3]

4 Architecture and Working Principle

Figure 1 combines the final ALB and browser evidence with the configured components. The browser screenshots show both page responses at the ALB DNS name. The target-group review nevertheless shows zero targets at that intermediate stage, and the final registration state is not captured.



Network context: VPC `Lab-new-vpc` (`10.0.0.0/16`); ALB subnets in `ap-south-1a` and `ap-south-1b`; both backend instances in `ap-south-1b`. Health check: HTTP `/`, traffic port 80, interval 5 seconds in the final review (screenshots 18–19 show the earlier 2-second entry).

Figure 1: Reconstructed architecture. Solid arrows show the evidenced request path and dashed arrows show the configured health-check path. The target-group review showed zero targets even though subsequent browser tests returned both backend responses.

For this flow to work, the security groups must permit HTTP on TCP port 80 from the client to the ALB and from the ALB to the EC2 targets, including health checks. Screenshots 23 and 24 show backend ingress rules that use the ALB security group as source. The ALB's own creation form showed no inbound rules; the later browser results establish endpoint reachability at test time, but the final ALB inbound rule is not documented. The edit screens also do not show a save confirmation for the backend rules.[7, 3]

When fully configured, a request would follow these steps:

1. A browser resolves the ALB DNS name and connects to its public-facing IPv4 endpoint over HTTP on port 80.
2. The ALB's port-80 listener receives the request and applies its default forwarding action.
3. The action forwards the request to target group `Target1`.
4. The ALB selects a healthy registered instance and sends the HTTP request to that target on port 80.
5. The instance's web server returns its page; the ALB relays the response to the browser. Separately, ALB health checks request `/` from targets on the configured health-check port.

The final selection of a target is controlled by the ALB and its target-group configuration; two observed

responses do not imply equal shares or strict alternation.[2, 6]

5 Implementation: Evidence-Based Sequence

The sequence follows the numbered screenshots and distinguishes completed results from intermediate forms. Each console screenshot is preserved as supplied; captions map the report figure back to its source screenshot.

5.1 Step 1: Confirm the VPC and subnet layout

Purpose. Identify the network that will contain the ALB and the backend instances.

Console navigation. Amazon VPC console → Your VPCs → select `Lab-new-vpc` → Resource map.

Observed configuration. Screenshot 1 shows VPC `Lab-new-vpc`, CIDR `10.0.0.0/16`, two subnets in `ap-south-1a` and `ap-south-1b`, route tables, an internet gateway connection, and the `Lab-new-vpc-s3` endpoint. The later ALB form selects two public subnets and confirms an IGW route. The VPC is existing infrastructure; these images do not show it being created during this practical.

Expected outcome. A VPC and public subnet choices are identified for the ALB and EC2 instances.[8, 3]

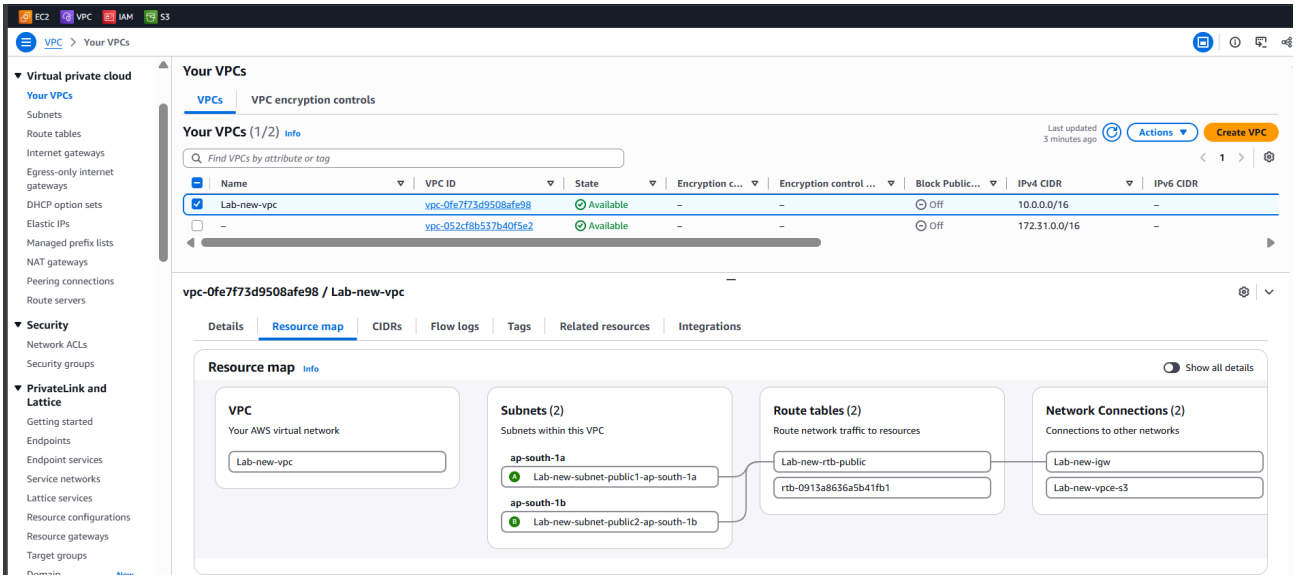


Figure 2: Source screenshot 1: VPC resource map for `Lab-new-vpc`.

5.2 Step 2: Create the ALB security group

Purpose. Establish a security-group identity for the load balancer.

Console navigation. Amazon EC2 console → Network & Security → Security Groups.

Observed configuration. Screenshot 2 confirms successful creation of security group `LoadBalancer` in `Lab-new-vpc`. Its ID is `sg-0317be9c3534379c1`. The same list contains `PublicServer1` and `PublicServer2`, but the success banner does not confirm their creation. The ALB group's inbound and outbound rules are detailed later in the ALB creation form.

Expected outcome. The ALB can be associated with this security group; rules must still allow client and backend traffic as appropriate.

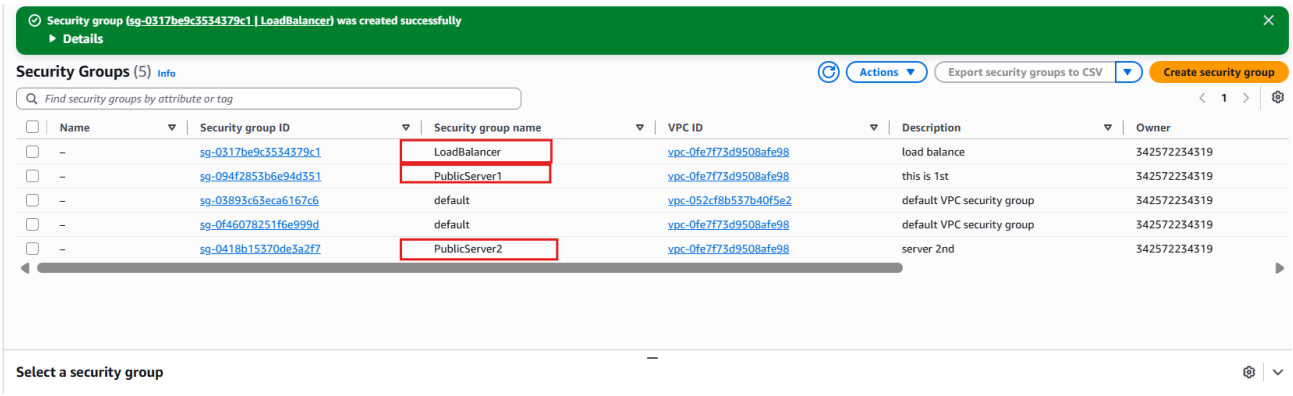


Figure 3: Source screenshot 2: successful creation of `LoadBalancer`; backend security groups are also listed.

5.3 Step 3: Configure the first EC2 web server

Purpose. Prepare a backend that serves a distinct page so its response can be identified during testing.

Console navigation. Amazon EC2 console → Instances → Launch instances.

Observed configuration. Screenshots 3–5 configure one instance named `Public1`, with Amazon Linux 2023, instance type `t3.micro`, one 8-GiB volume, subnet `Lab-new-subnet-public2-ap-south-1b` (CIDR `10.0.16.0/20`), public-IP auto-assignment enabled, and security group `PublicServer1` (ID `sg-094f2853b6e94d351`). The user-data script installs Nginx, enables and starts it, and writes the unique text to its web root:

```
#!/bin/bash
yum install nginx -y
systemctl enable --now nginx
cd /usr/share/nginx/html
echo "This is Server1" > index.html
```

These are launch-form values; the later running-instance screen provides evidence that the instance was launched.

Expected outcome. The instance should serve “This is Server1” over HTTP after the user-data commands complete.

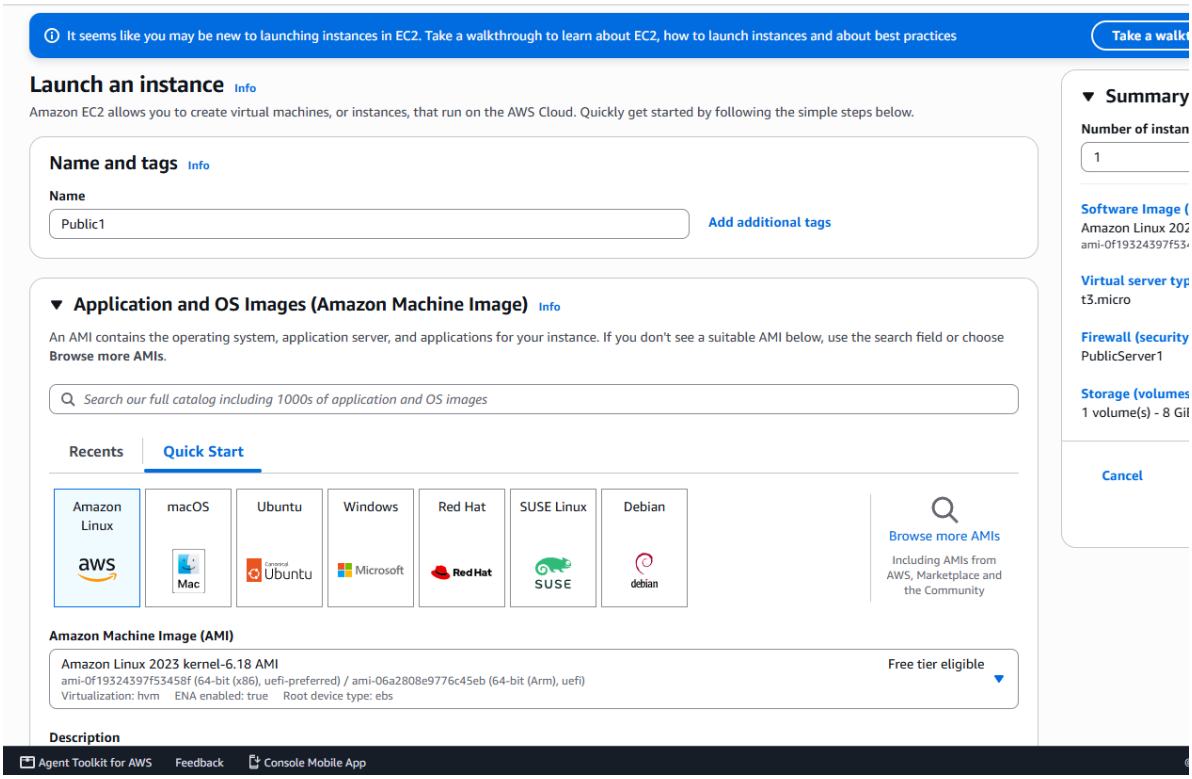


Figure 4: Source screenshot 3: first instance name and Amazon Linux 2023 image selection.

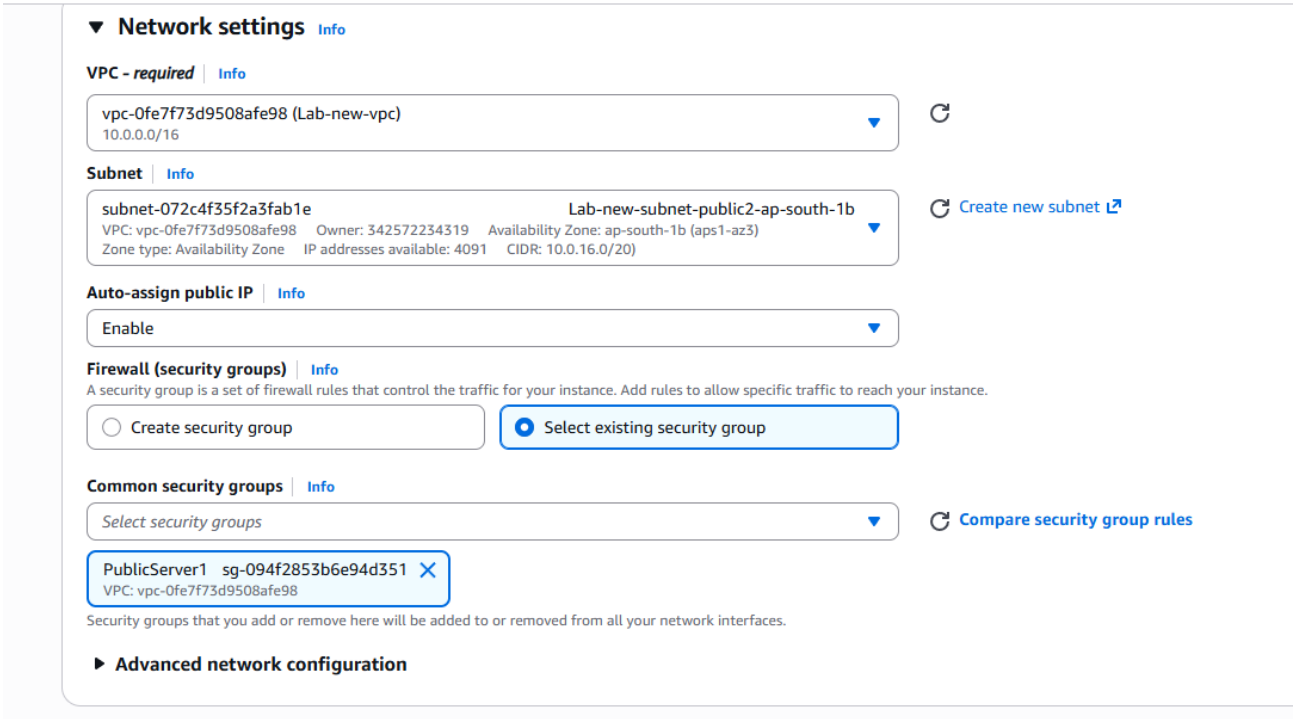


Figure 5: Source screenshot 4: first instance subnet, public-IP setting, and `PublicServer1` security group.

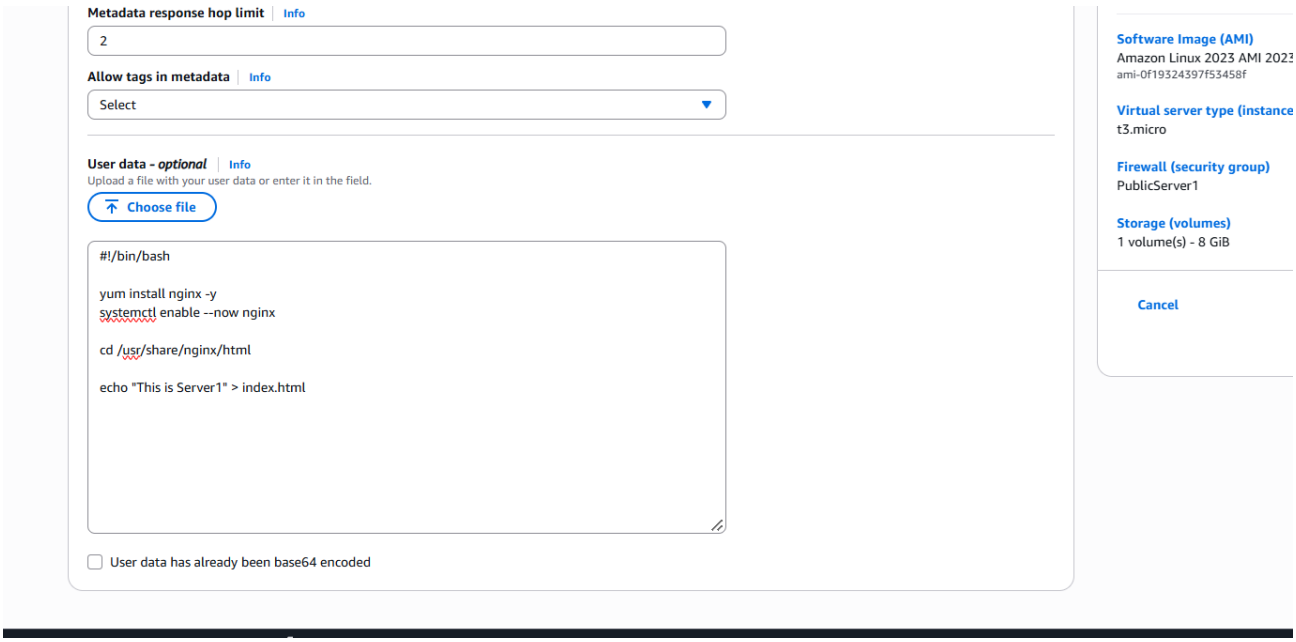


Figure 6: Source screenshot 5: Nginx installation and Server1 page user data.

5.4 Step 4: Configure the second EC2 web server

Purpose. Create a second backend with a different response body.

Console navigation. EC2 console → Instances → Launch instances.

Observed configuration. Screenshots 6–8 configure one instance named `Public2`, also using Amazon Linux 2023, `t3.micro`, and one 8-GiB volume. Screenshot 7 selects subnet `Lab-new-subnet-public2-ap-south-1b`, enables public-IP assignment, and uses security group `PublicServer2` (ID `sg-0418b153 70de3a2f7`). Its user data installs and starts Nginx and writes "This is Server2" to `/usr/share/nginx/html/index.html`. Screenshot 8 also displays the console's IMDSv2 metadata-token warning; it is not part of the web-server setup itself.

Expected outcome. The second instance should return a distinct Server2 page, allowing a response to be attributed to this backend.

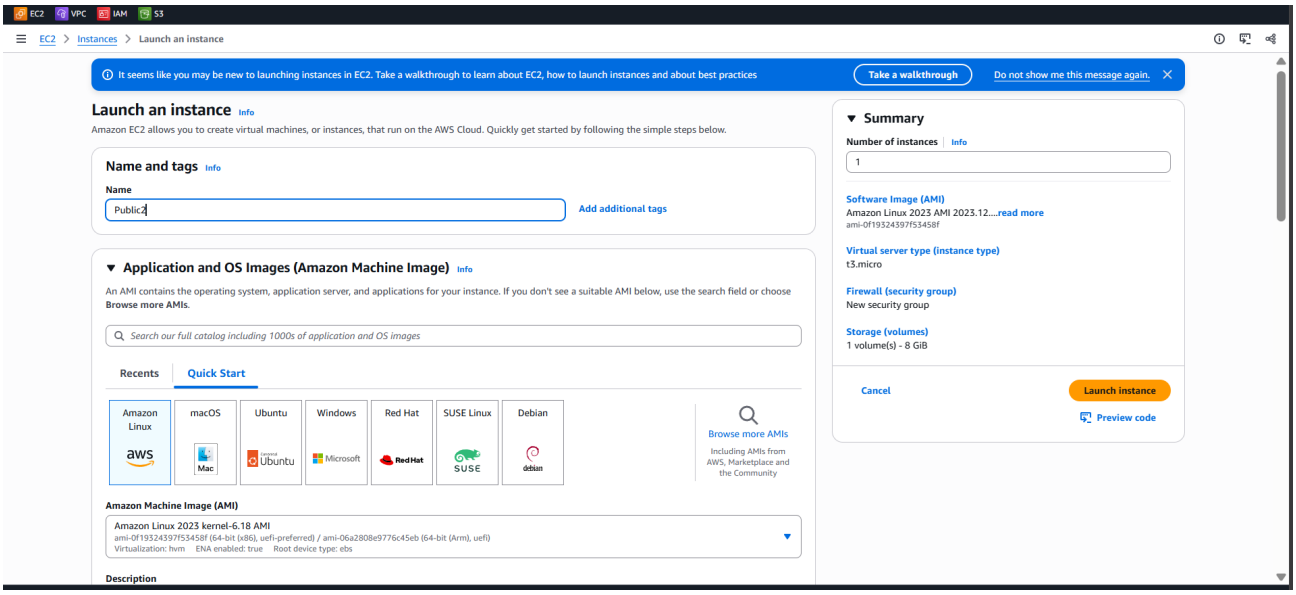


Figure 7: Source screenshot 6: second instance launch form, name `Public2`.

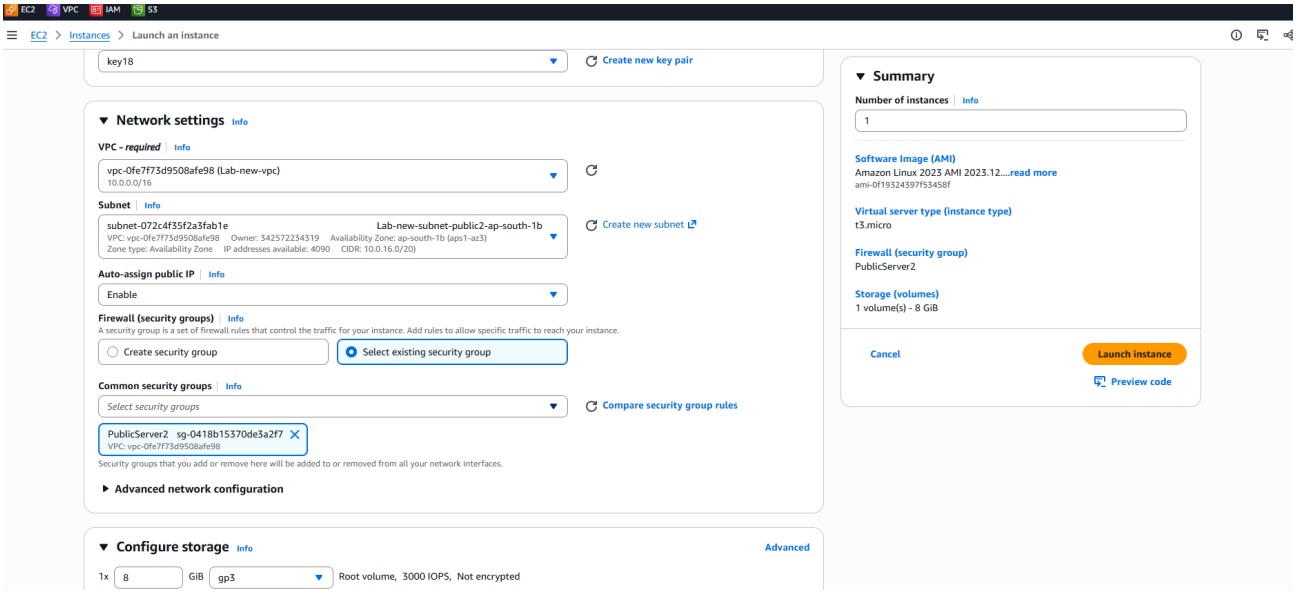


Figure 8: Source screenshot 7: second instance network and `PublicServer2` security-group selection.

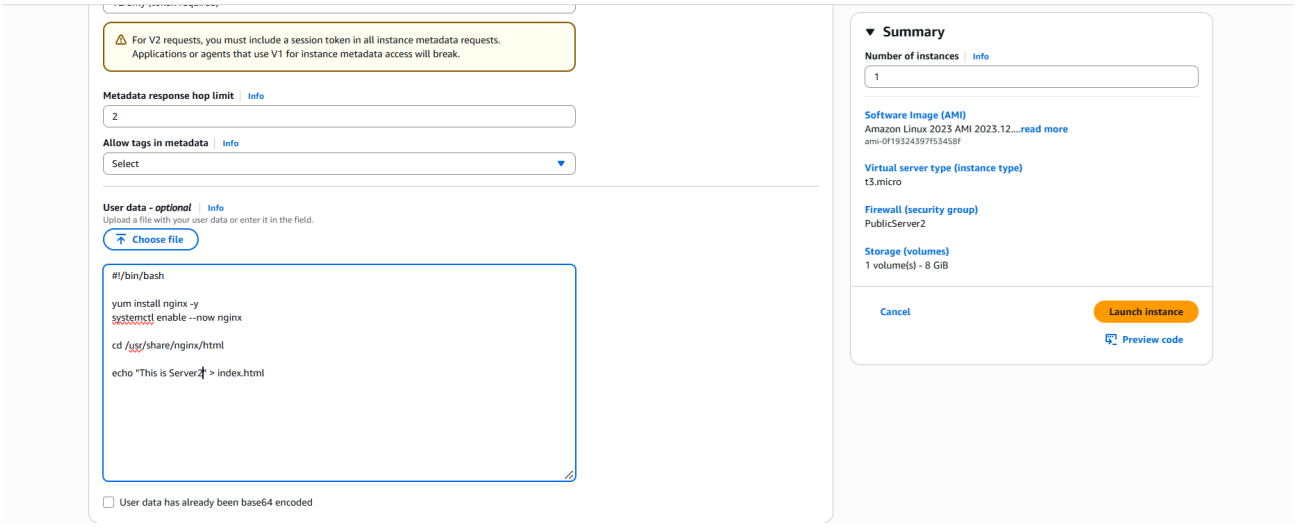


Figure 9: Source screenshot 8: Nginx installation and Server2 page user data.

5.5 Step 5: Confirm the initial load-balancer state

Purpose. Establish that the load balancer is created later in the sequence rather than pre-existing.

Console navigation. EC2 console → Load Balancers.

Observed state. Screenshot 9 shows an empty load-balancer list in `ap-south-1` and the Create load balancer control.

Expected outcome. Continue to the ALB creation workflow.

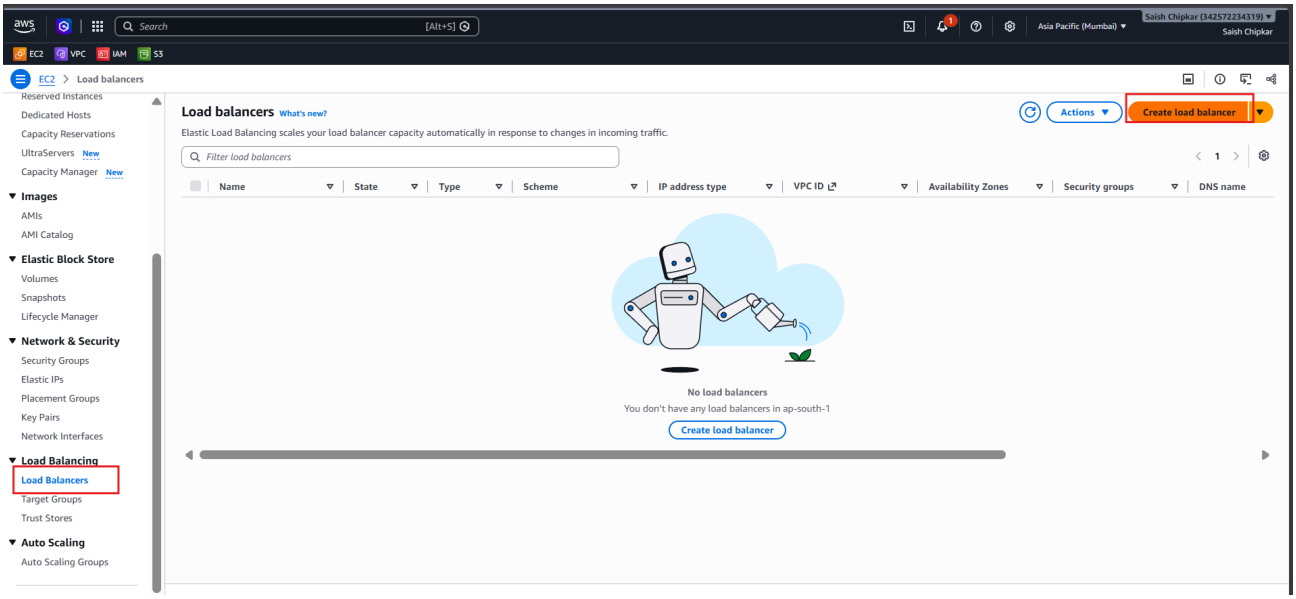


Figure 10: Source screenshot 9: no load balancers are listed before this practical's ALB setup.

5.6 Step 6: Select and name the Application Load Balancer

Purpose. Choose the ALB service for HTTP/HTTPS application traffic and set its externally facing identity.

Console navigation. EC2 console → Load Balancers → Create load balancer → Application Load Balancer.

Observed configuration. Screenshot 10 highlights the Application Load Balancer option, not Network or Gateway Load Balancer. Screenshot 11 sets name `LoadBalance`, scheme `Internet-facing`, and address type `IPv4`. These are the basic form settings; screenshot 25 later confirms the created ALB.

Expected outcome. An ALB creation form ready for VPC and subnet mapping.[2, 3]

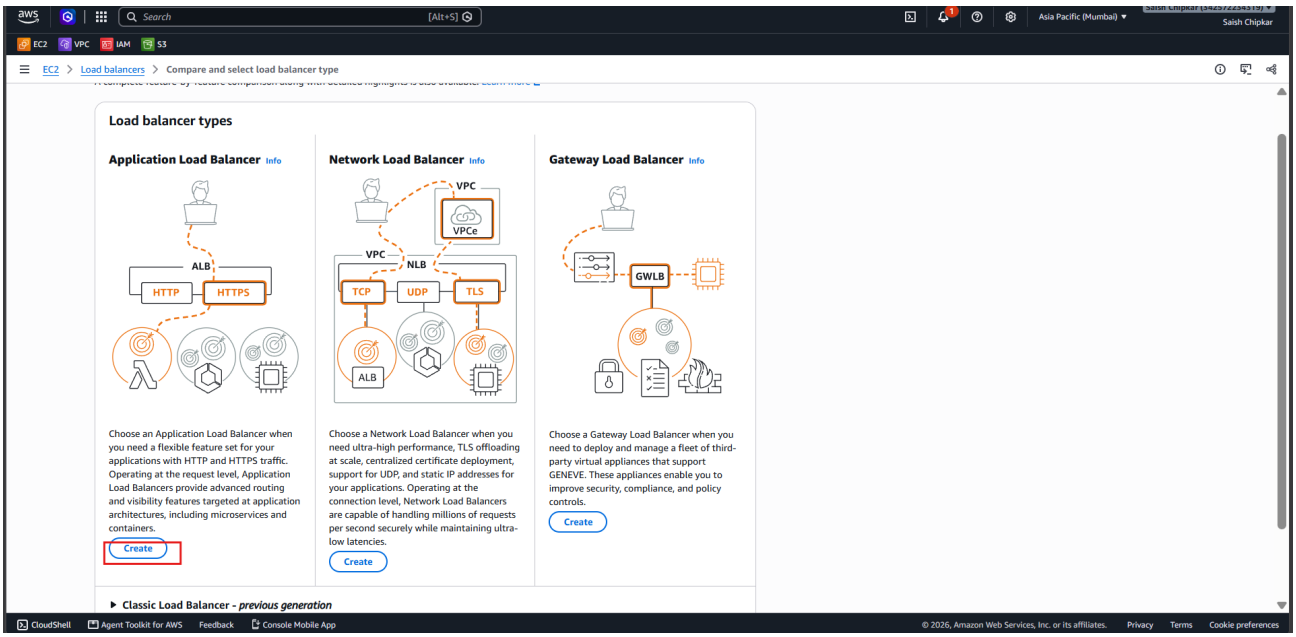


Figure 11: Source screenshot 10: Application Load Balancer selected rather than NLB or GWLB.

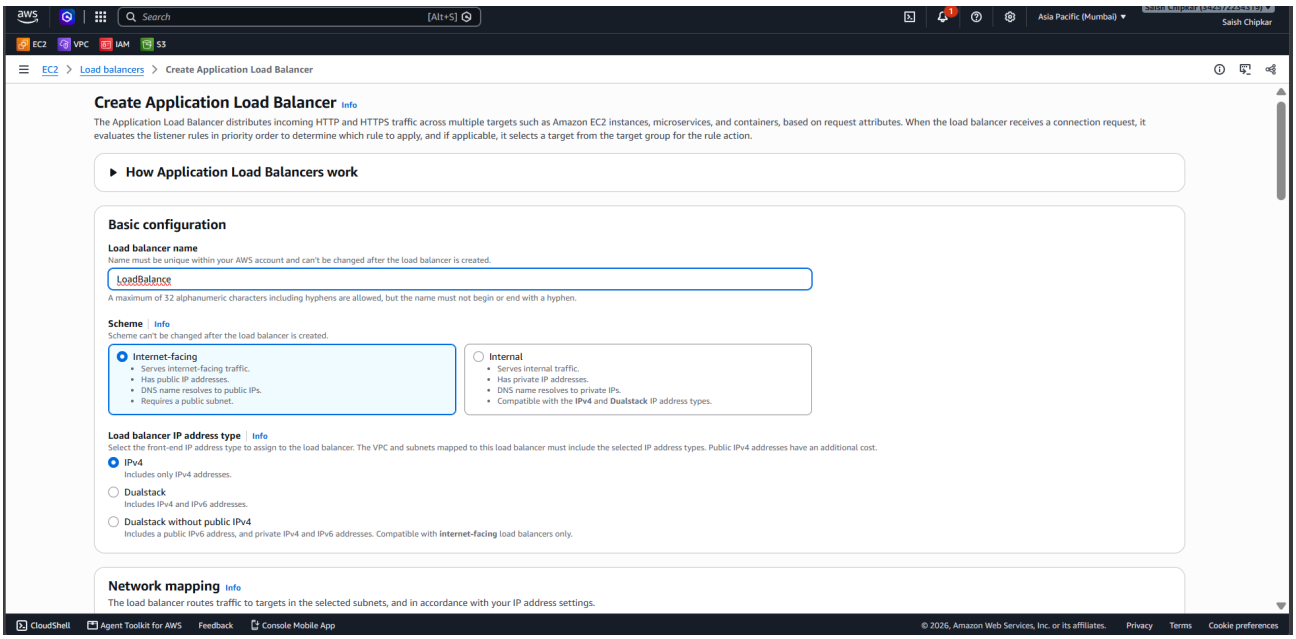


Figure 12: Source screenshot 11: ALB name `LoadBalance`, Internet-facing scheme, and IPv4 address type.

5.7 Step 7: Map subnets and select the ALB security group

Purpose. Place ALB nodes in two zones and associate the ALB firewall.

Console navigation. ALB creation form → Network mapping and Security groups.

Observed configuration. Screenshot 12 selects VPC `Lab-new-vpc` (`10.0.0.0/16`) and these subnets:

- `Lab-new-subnet-public1-ap-south-1a` (`10.0.0.0/20`);
- `Lab-new-subnet-public2-ap-south-1b` (`10.0.16.0/20`).

Both show an IGW route. Screenshot 13 selects security group `LoadBalancer`, showing zero inbound rules, one outbound rule, and a reachability warning. The shown configuration therefore lacks an evidenced client-facing HTTP ingress rule at this point.[3, 7]

Expected outcome. The ALB is configured to use subnets in two AZs. The ALB security group must allow incoming HTTP traffic for the browser test; its final inbound rule is not shown.

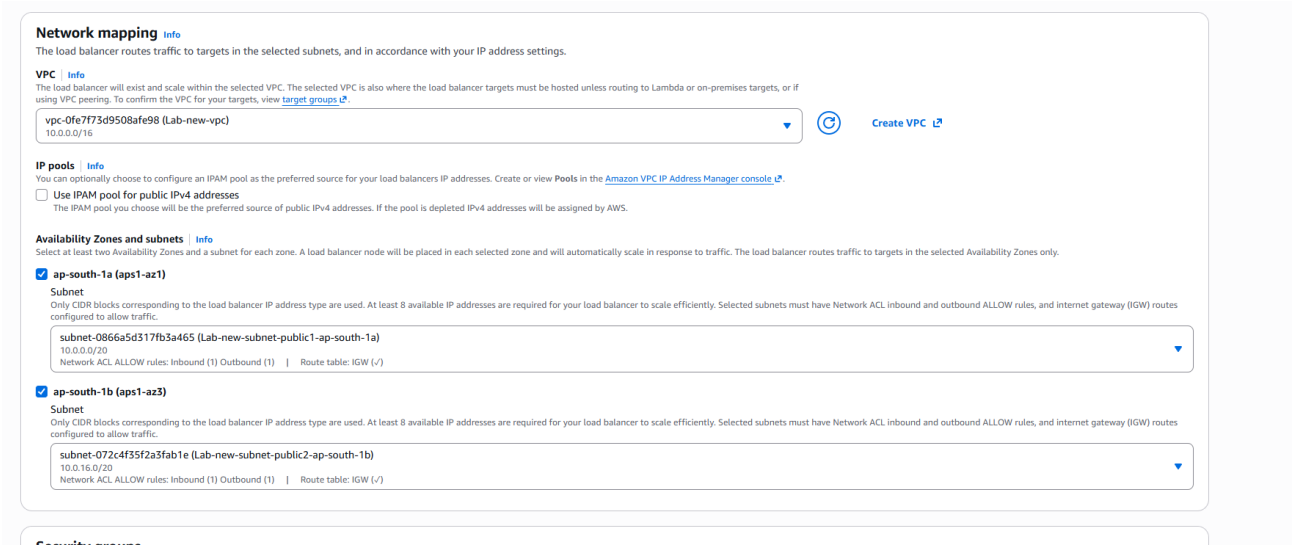


Figure 13: Source screenshot 12: selected VPC and public subnets in `ap-south-1a` and `ap-south-1b`.

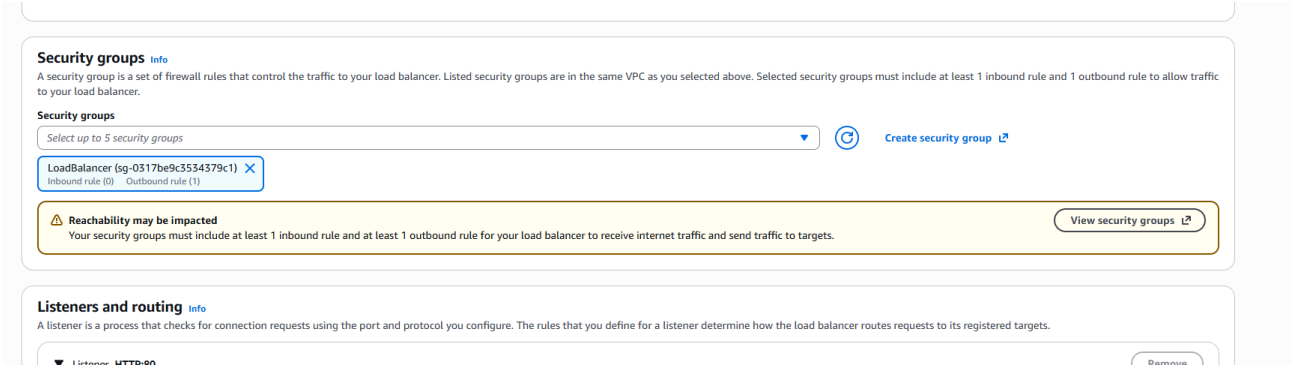


Figure 14: Source screenshot 13: selected ALB security group with zero inbound rules and a reachability warning.

5.8 Step 8: Configure the initial HTTP listener

Purpose. Define how client HTTP requests enter the ALB and where they will be forwarded.

Console navigation. ALB creation form → Listeners and routing.

Observed configuration. Screenshot 14 sets protocol `HTTP`, port `80`, and default action "Forward to target groups." The target group selector is blank in this intermediate form, so the relationship is completed later after the target group exists.

Expected outcome. An HTTP listener will receive requests on port 80; it needs a target group selected as its forwarding destination.

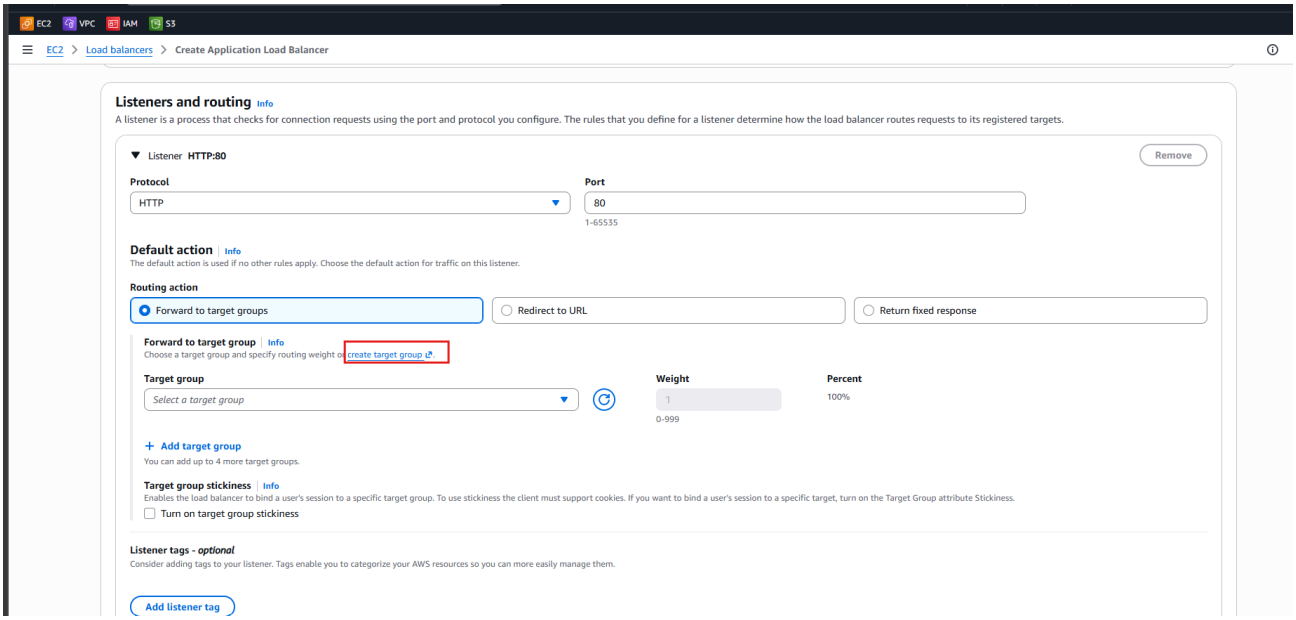


Figure 15: Source screenshot 14: HTTP:80 listener before a target group is selected.

5.9 Step 9: Create and configure target group `Target1`

Purpose. Define a logical destination set for the EC2 web servers and the health criteria used to assess them.

Console navigation. EC2 console → Target Groups → Create target group.

Observed configuration. Screenshots 15–16 select target type `Instances`, name `Target1`, protocol `HTTP`, port `80`, target IP type `IPv4`, VPC `Lab-new-vpc`, and protocol version `HTTP1`. Screenshots 17–19 set health checks to HTTP path `/`, traffic port, timeout 2 seconds, healthy threshold 5, unhealthy threshold 2, and success code `200`. The interval is entered as 2 seconds in screenshots 18–19 despite a displayed allowed range of 5–300 seconds. The later review screen shows it changed to 5 seconds, so the 2-second value is an intermediate invalid entry, not the final reviewed value. The optional Target optimizer remains `Off - Default`.

Expected outcome. The target group has HTTP/1 traffic settings and valid final health-check settings. Target health itself is not displayed. [5, 6]

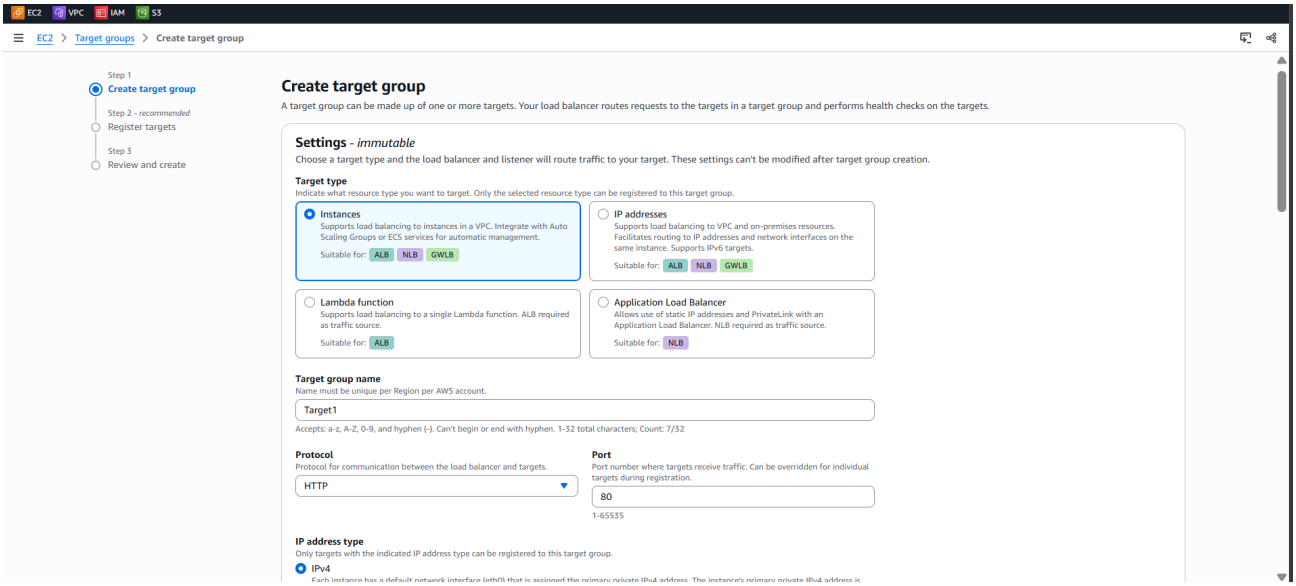


Figure 16: Source screenshot 15: target type `Instances`, name `Target1`, HTTP port 80.

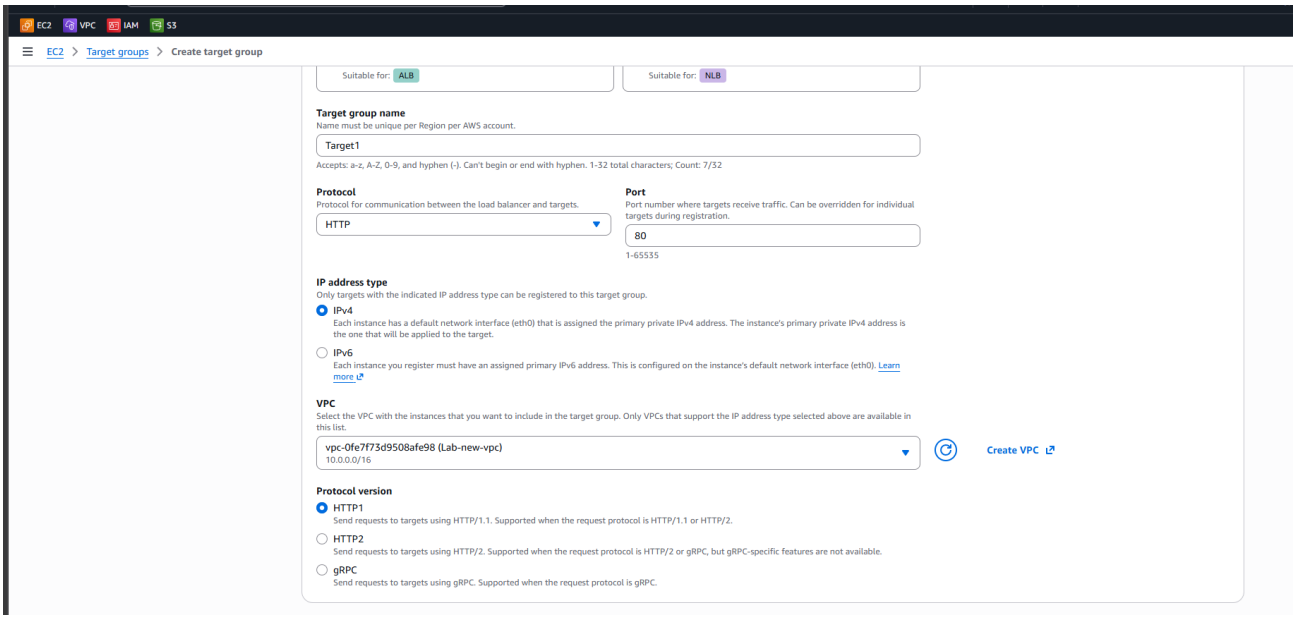


Figure 17: Source screenshot 16: target-group VPC, IPv4, and HTTP/1 protocol version.

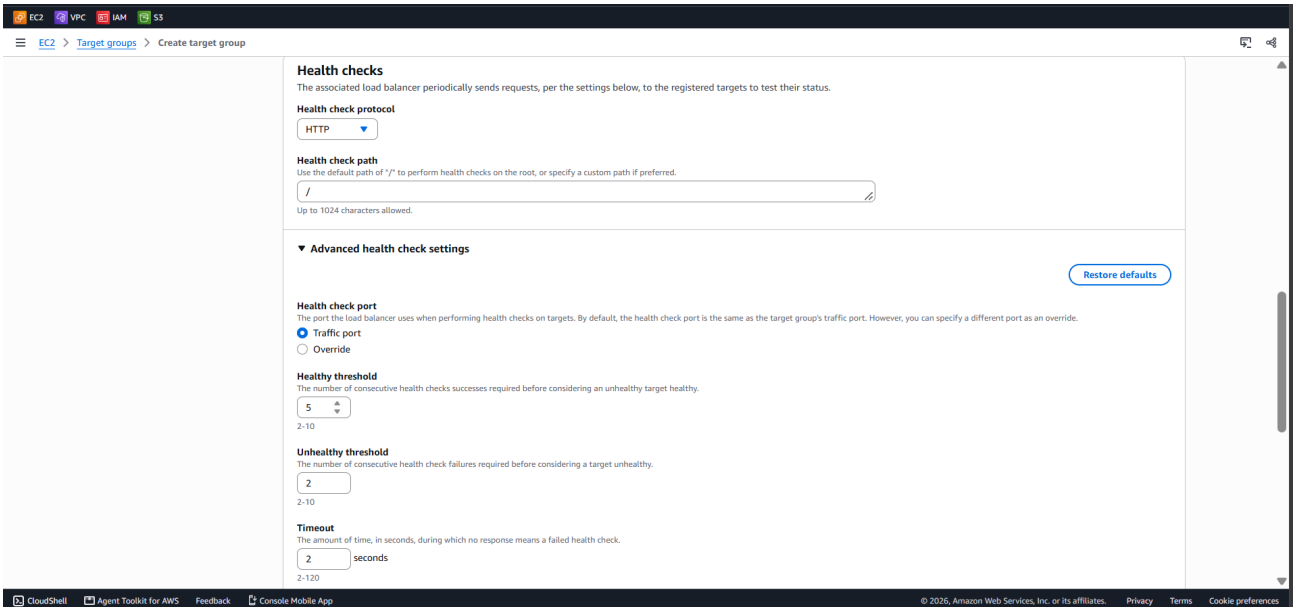


Figure 18: Source screenshot 17: HTTP health-check path, traffic port, thresholds, and timeout.

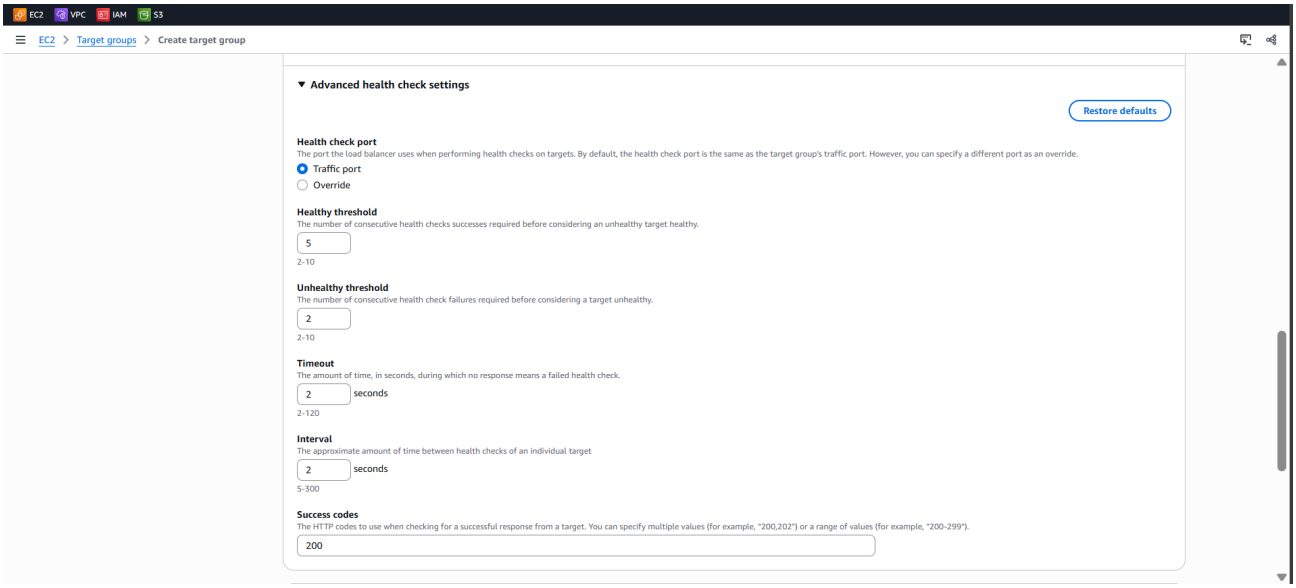


Figure 19: Source screenshot 18: intermediate interval 2 seconds (outside the displayed 5–300-second range), success code 200.

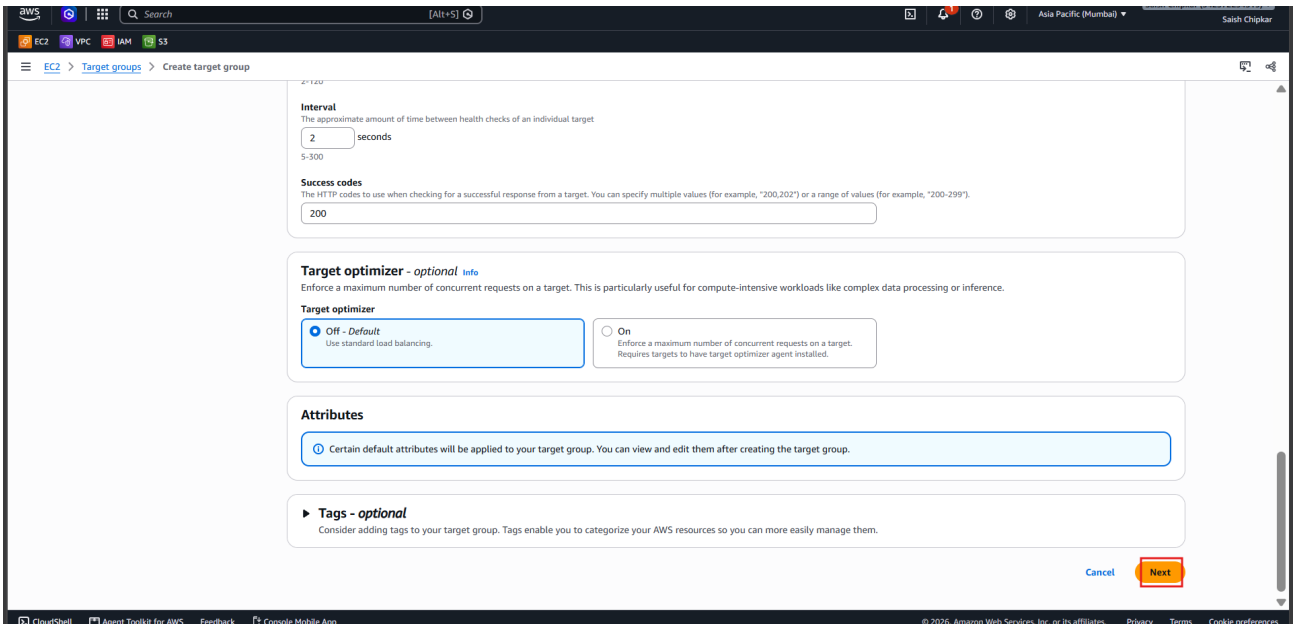


Figure 20: Source screenshot 19: the interval remains 2 seconds in this form; Target optimizer is off by default.

5.10 Step 10: Select and review backend targets

Purpose. Choose the EC2 instances to register with `Target1`.

Console navigation. Target-group creation wizard → Register targets → Review and create.

Observed configuration and discrepancy. Screenshot 20 lists both instances as running and selected on port 80: `Public2` (`i-0885c0fc45d5e47cd`, private IPv4 `10.0.16.116`) and `Public1` (`i-0d7255b1d386a5568`, private IPv4 `10.0.28.29`). Both are shown in `ap-south-1b`. The form still shows an "Include as pending below" button and its review table has zero targets at that moment. Screenshot 21 confirms `Targets (0)` and `No targets added`; it also shows the final interval of 5 seconds and the Create target group control. Thus, the screenshots show selection attempts, not successful registration of these two targets. Screenshot 22 later shows that `Target1` exists and is selectable, while the subsequent browser tests return both page bodies. No later target-list or target-health screen explains the intermediate zero-target review.

Expected outcome. A completed registration should list both instances with port 80 in the target group. That final list is not included in the screenshots; the later browser tests provide separate evidence of both backend responses through the ALB endpoint.

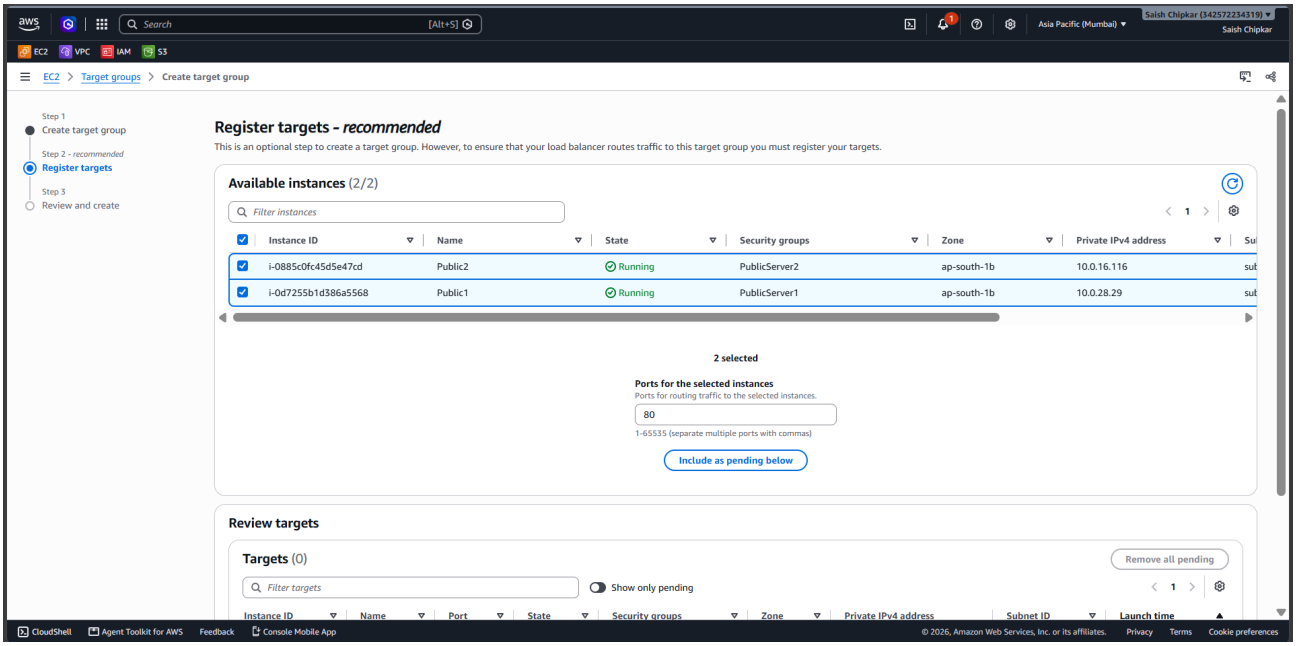


Figure 21: Source screenshot 20: both running EC2 instances are selected for port 80, but not yet included in the pending target list.

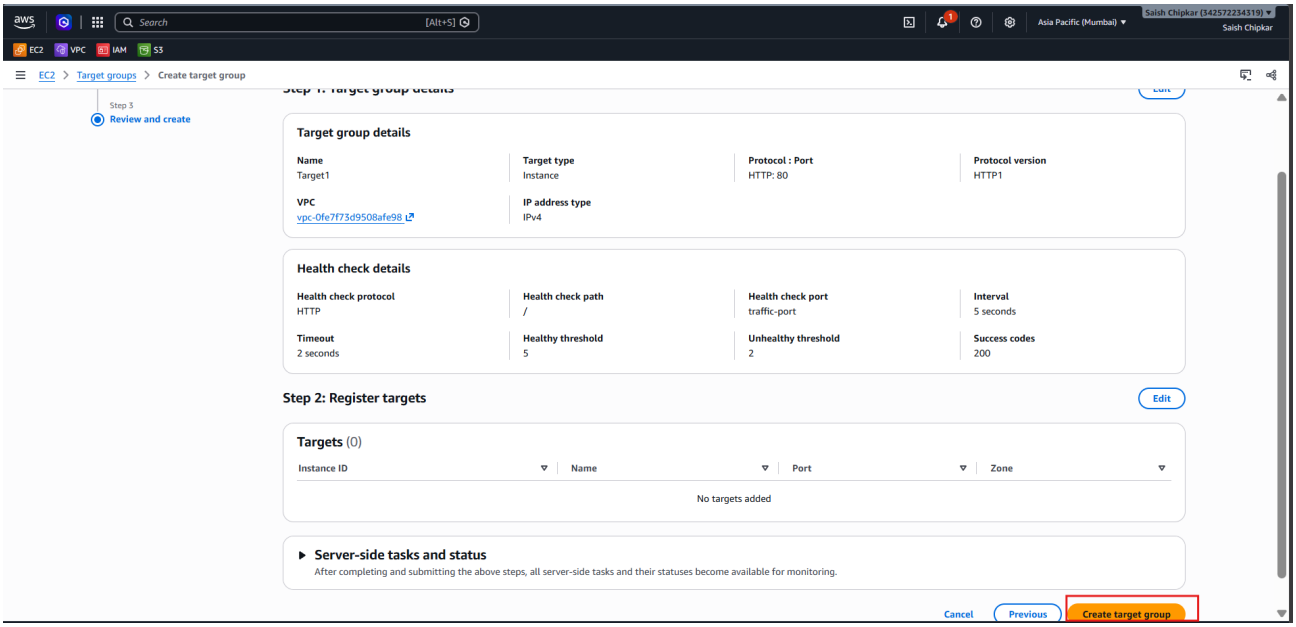


Figure 22: Source screenshot 21: target-group review shows zero registered targets and the interval corrected to 5 seconds.

5.11 Step 11: Associate the target group and complete ALB creation

Purpose. Connect the HTTP listener to the target group and make the ALB available to clients.

Console navigation. Return to the ALB creation form → Listeners and routing → select a target group, then complete ALB creation.

Observed configuration. Screenshot 22 shows listener `HTTP:80` with "Forward to target groups" and `Target1` selected. The target group weight is 1 (100%), and target-group stickiness is off. This weight is for the selected target group; it is not evidence of equal distribution among its instances. Screenshot 25 later shows `LoadBalance` in state `Active`, confirming the ALB was created successfully.

Expected outcome. The active ALB exposes its DNS hostname and routes HTTP requests according to its listener action.

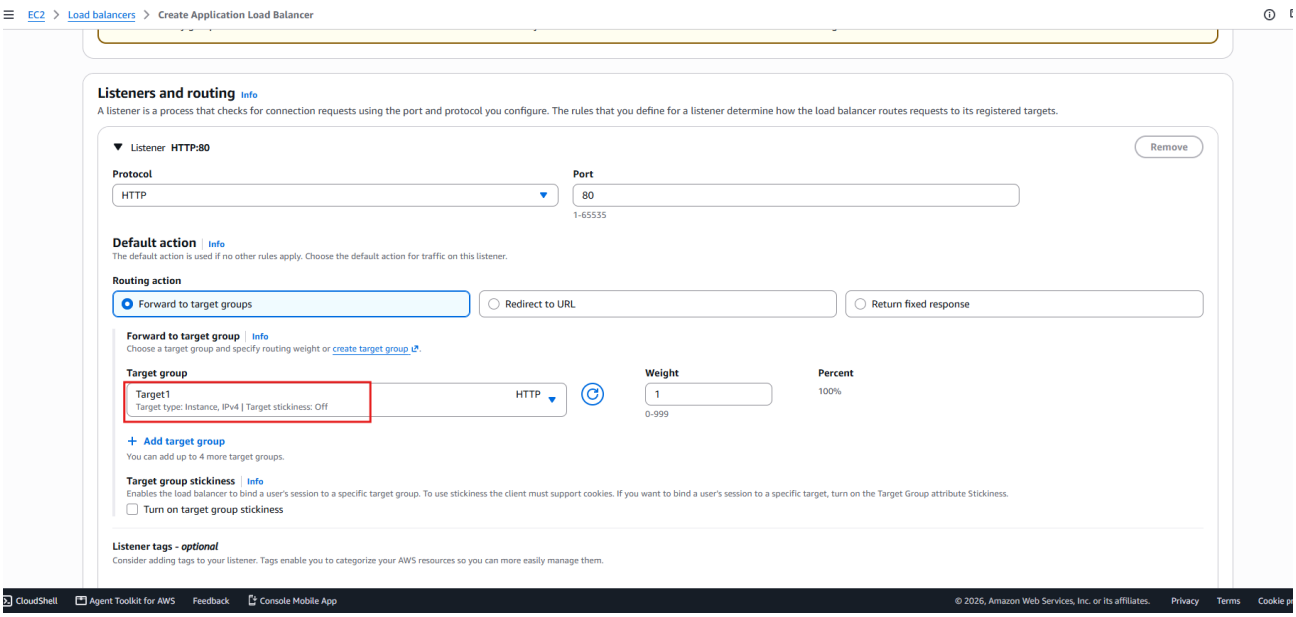


Figure 23: Source screenshot 22: final listener configuration forwards HTTP:80 traffic to `Target1`.

5.12 Step 12: Configure backend security-group ingress

Purpose. Permit the load balancer to reach each web server on its HTTP traffic and health-check port without opening the backend port to arbitrary sources.

Console navigation. EC2 console → Network & Security → Security Groups → edit inbound rules for each backend security group.

Observed configuration. Screenshot 23 edits the inbound rule for `PublicServer1` (`sg-094f2853b6e94d351`); screenshot 24 edits `PublicServer2` (`sg-0418b15370de3a2f7`). Both forms specify type `HTTP`, protocol `TCP`, port `80`, and custom source security group `LoadBalancer` (`sg-0317be9c3534379c1`). The Save rules button is visible in each image, so the captures show the proposed rule values but not a save confirmation. The ALB security group's final inbound rule is also not shown. The explanatory callouts beside the source fields name the other instance's security group, but the actual source ID in both forms matches `LoadBalancer` from screenshot 2; the source ID and edited-group breadcrumb are the reliable identifiers.[7]

Expected outcome. When saved, the backend groups accept port-80 connections originating from the ALB's security group, including health checks.

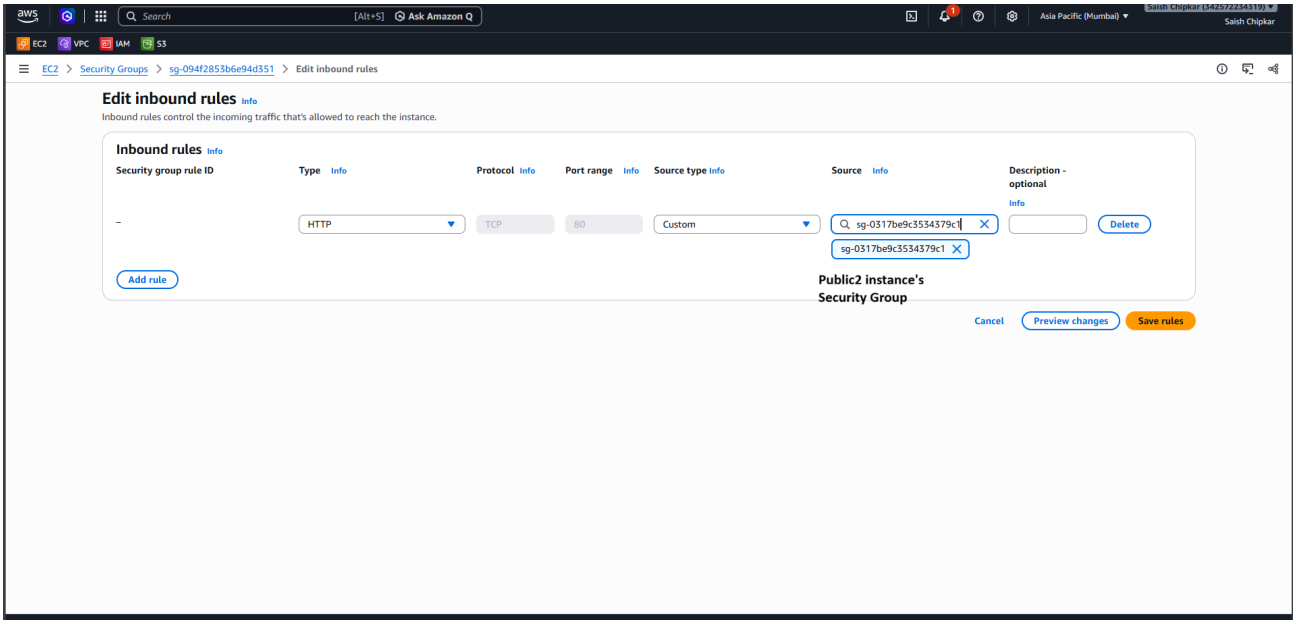


Figure 24: Source screenshot 23: proposed HTTP/TCP 80 ingress from the ALB security group to `PublicServer1`.

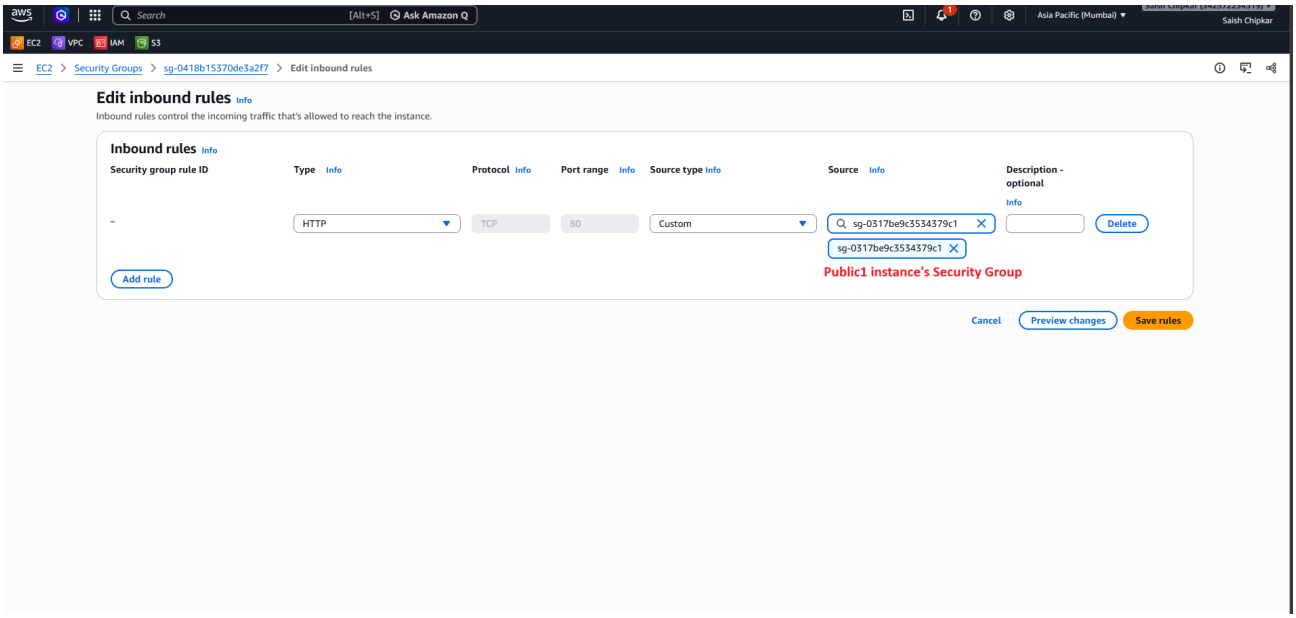


Figure 25: Source screenshot 24: proposed HTTP/TCP 80 ingress from the ALB security group to `PublicServer2`.

5.13 Step 13: Verify ALB creation and obtain its DNS name

Purpose. Confirm the resource is active and identify the single client-facing endpoint.

Console navigation. EC2 console → Load Balancers → select `LoadBalance`.

Observed result. Screenshot 25 shows load balancer type `application`, state `Active`, scheme `Internet-facing`, IP type `IPv4`, VPC `Lab-new-vpc`, and two Availability Zones. Its DNS name is `LoadBalance-1426977249.ap-south-1.elb.amazonaws.com`. The console shows creation on November 2, 2026. This is direct evidence of successful ALB creation; the screenshot does not show target health.

Expected outcome. The DNS name can be used as the browser endpoint instead of addressing either EC2 instance directly.

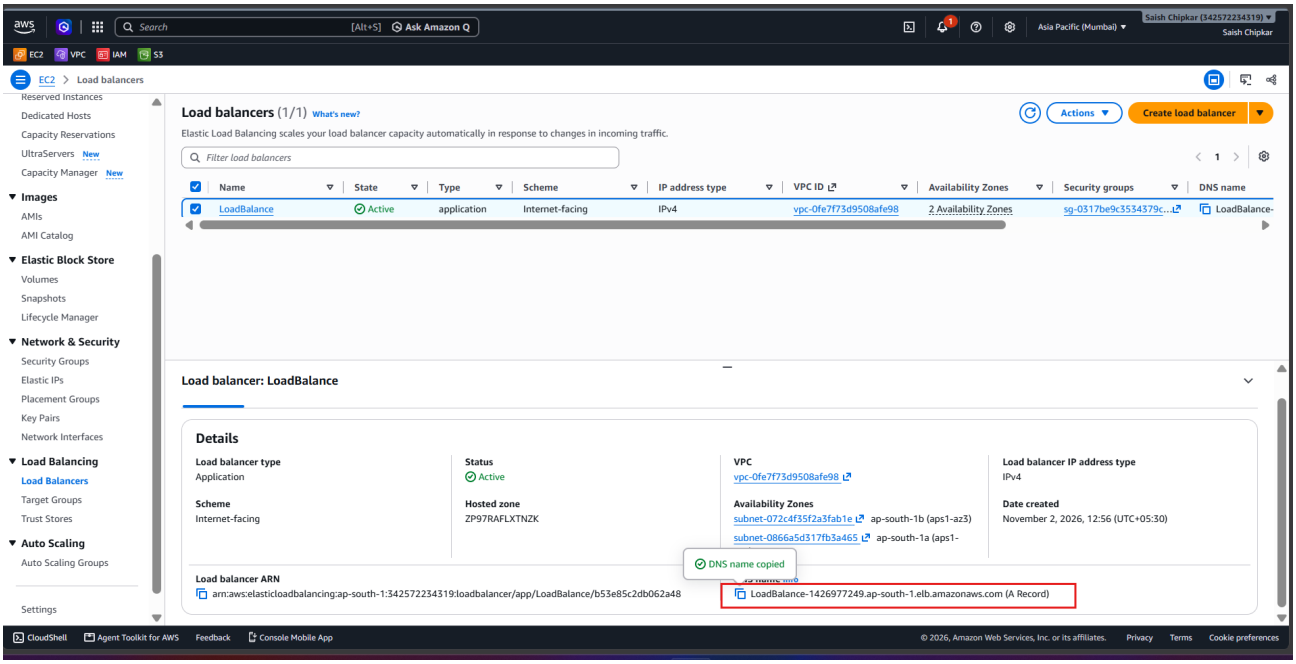


Figure 26: Source screenshot 25: active internet-facing ALB and DNS name.

5.14 Step 14: Test the ALB endpoint in a browser

Purpose. Verify that requests made to the ALB's DNS endpoint can return content from each configured web server.

Console navigation. Open the ALB DNS name in a web browser using HTTP.

Observed results. Screenshots 26 and 27 show the same ALB hostname in the browser address bar and return "This is Server1" and "This is Server2", respectively. The browser marks the connection "Not secure," consistent with the HTTP endpoint shown in the listener configuration; no HTTPS listener or TLS certificate is evidenced. The results establish that both distinct backend responses were observed through the ALB DNS endpoint at the captured times. They do not establish equal traffic shares, strict alternation, the number of requests made, or target health over time.

Expected outcome. A browser request to the shared ALB endpoint receives an HTTP response from a backend server. The two captures demonstrate both distinct page bodies, but no target-health dashboard is shown.

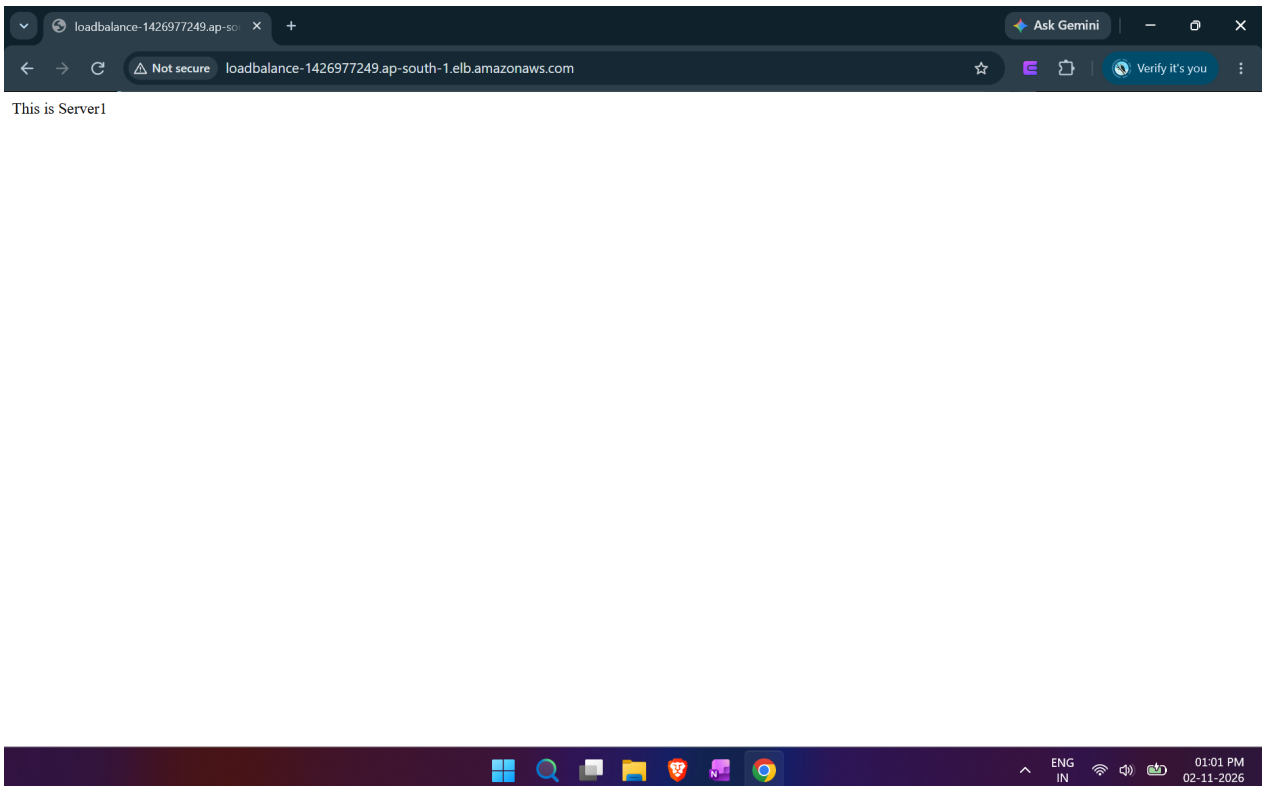


Figure 27: Source screenshot 26: request to the ALB DNS name returns “This is Server1”.



Figure 28: Source screenshot 27: request to the same ALB DNS name returns “This is Server2”.

6 Consolidated Configuration Details

Tables 1 and 2 record the values that can be read in the screenshots. "Not visible" means the available files do not establish a value; it does not mean that the resource or setting does not exist.

Setting	Screenshot evidence / status
AWS Region	Asia Pacific (Mumbai), `ap-south-1`.
Load balancer type	Application Load Balancer.
Load balancer name / state	`LoadBalance`; state `Active` in screenshot 25.
DNS name	`LoadBalance-1426977249.ap-south-1.elb.amazonaws.com`.
Scheme / IP type	Internet-facing; IPv4.
VPC	`Lab-new-vpc`, IPv4 CIDR `10.0.0.0/16`.
ALB subnets	`Lab-new-subnet-public1-ap-south-1a` (`10.0.0.0/20`) and `Lab-new-subnet-public2-ap-south-1b` (`10.0.16.0/20`); AZs `ap-south-1a` and `ap-south-1b`.
Listener	HTTP, port 80; default action forwards to `Target1` in screenshot 22.
Listener target-group weight	1 (100%); stickiness off. This is a target-group weight, not a per-instance distribution ratio.
Target group	`Target1`; target type `Instances`; IPv4; VPC `Lab-new-vpc`.
Target protocol / port	HTTP, port 80; protocol version `HTTP1`.
Health-check protocol / path / port	HTTP, `/`, `Traffic port`.
Health-check timeout	2 seconds.
Healthy / unhealthy thresholds	5 consecutive successes / 2 consecutive failures.
Health-check interval	Initially 2 seconds (screenshots 18–19); later review shows 5 seconds (screenshot 21), within the displayed 5–300-second range.
Success codes	`200`.
ALB security group	`LoadBalancer` (`sg-0317be9c3534379c1`); screenshot 13 shows 0 inbound rules and 1 outbound rule. Final inbound rules not shown.

Table 1: ALB and target-group values evidenced in the console screenshots.

Setting	Screenshot evidence / status
Backend instances	`Public1` (`i-0d7255b1d386a5568`, private IPv4 `10.0.28.29`) and `Public2` (`i-0885c0fc45d5e47cd`, private IPv4 `10.0.16.116`); both shown Running in `ap-south-1b`.
Instance configuration	Amazon Linux 2023; `t3.micro`; one 8-GiB volume per launch form.
Backend network	Both launch forms select `Lab-new-subnet-public2-ap-south-1b` and enable public-IP assignment.
Web application setup	User data installs and enables Nginx; first page is "This is Server1" and second is "This is Server2".
Backend security groups	`PublicServer1` (`sg-094f2853b6e94d351`) and `PublicServer2` (`sg-0418b15370de3a2f7`).
Backend ingress rule	Both rule editors show HTTP/TCP port 80 with source `LoadBalancer`; Save rules remains visible, so saved state is not directly confirmed.
Target registration	Both instances selected in screenshot 20; screenshot 21 review states `Targets (0)`. No subsequent target-list or health screenshot.
Browser test	ALB DNS endpoint returns "This is Server1" and "This is Server2" in screenshots 26–27.

Table 2: Backend configuration and unresolved state details.

7 Testing and Verification

Screenshots 25–27 provide direct evidence of an active ALB and two browser responses. The address bar in both browser captures shows `loadbalance-1426977249.ap-south-1.elb.amazonaws.com`, matching the ALB DNS name shown in the console, and the response bodies differ: "This is Server1" and "This is Server2". This demonstrates that requests made to the same ALB hostname returned content from each backend at least once in the captured tests. The pages also show the browser's "Not secure" indicator, consistent with HTTP rather than HTTPS.

These observations do not show the number or cadence of requests, so they do not prove equal distribution or strict alternation. They also do not establish that both targets were continuously healthy or how the target group reached a registered state: screenshot 21 reports zero targets, and no later target registration or health view is present. Likewise, the ALB security group had zero inbound rules in its creation form, and the final inbound rule is not shown, although the browser captures demonstrate that the endpoint was reachable when tested.

A fuller operational check would retain evidence of both targets registered and reporting healthy, verify the final ALB inbound rule and both backend security-group rules, then make repeated requests through the DNS name while recording responses. This would help distinguish successful end-to-end responses from target health over time or a particular balancing algorithm.

8 Results and Observations

The supplied screenshots support the following observations:

- The ALB `LoadBalance` was created in `ap-south-1` and is shown as Active, internet-facing, and IPv4, using two selected public subnets in separate AZs.
- The HTTP:80 listener is shown forwarding to `Target1`; the target group uses instance targets, HTTP:80, and HTTP/1.
- The initial health-check interval entry of 2 seconds was corrected to 5 seconds by the target-group review. Other final visible values are HTTP path `/`, traffic port, timeout 2 seconds, thresholds 5/2, and success code 200.
- The browser screenshots show both distinct backend pages at the ALB DNS name, supporting successful end-to-end HTTP requests through the shared endpoint.
- Target registration remains inconsistent in the evidence: two instances are selected in screenshot 20, but the review in screenshot 21 shows zero targets. No final target-health view is supplied.
- Security-group rule editors show backend HTTP/TCP 80 ingress from the ALB group, but not save confirmation. The ALB group form shows zero inbound rules, and the final ALB ingress configuration is not shown.

Taken together, the evidence confirms successful ALB creation and working browser responses from both backend page contents through the ALB DNS endpoint. It does not certify the final target registration or target health, and the firewall screenshots do not document the final effective rules. These limits are distinct from the successful outcomes that are directly visible.

9 Conclusion

This practical configured two Amazon Linux 2023 EC2 web servers, an instance target group, and an internet-facing IPv4 Application Load Balancer with an HTTP:80 listener. The ALB was created successfully and shown as Active. The browser screenshots use its DNS name and return "This

is Server1" and "This is Server2", demonstrating that the shared endpoint returned content from both backend servers. The final health-check review shows a valid 5-second interval after an earlier out-of-range 2-second entry.

The evidence also has unresolved details that should be retained in any technical handoff: the target-group review says zero targets despite the earlier selection of two instances and later browser responses, and the final security-group rule state is not documented. No target-health screen or repeated-request measurements establish ongoing health or distribution behavior. The practical therefore demonstrates the ALB request path and distinct backend responses, while also illustrating why final target and firewall verification should be captured before closing an implementation.

10 Glossary

ALB Application Load Balancer; an Elastic Load Balancing service for application-layer HTTP and HTTPS traffic.

Availability Zone A distinct location within an AWS Region; a subnet is placed in one AZ.

EC2 instance A virtual server provided by Amazon Elastic Compute Cloud.

Health check A periodic request used by a load balancer to assess whether a registered target responds as configured.

HTTP Hypertext Transfer Protocol, the application protocol used for the listener, target traffic, and health checks shown here.

Listener A load balancer process that accepts connections on a configured protocol and port and applies routing rules.

Security group A stateful virtual firewall that controls allowed inbound and outbound traffic for an associated resource.

Target A destination registered in a target group; here, the selected target type is an EC2 instance.

Target group A set of registered targets with shared routing and health-check configuration.

TCP port A numbered transport endpoint. Port 80 is the configured conventional HTTP endpoint in this practical.

VPC Virtual Private Cloud; a logically isolated network for AWS resources.

References

[1] Amazon Web Services, "What is Amazon EC2?" *Amazon EC2 User Guide*. https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html Accessed November 3, 2026.

[2] Amazon Web Services, "What is an Application Load Balancer?" *Elastic Load Balancing User Guide*. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html Accessed November 3, 2026.

[3] Amazon Web Services, "Create an Application Load Balancer." *Elastic Load Balancing User Guide*. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/create-application-load-balancer.html Accessed November 3, 2026.

[4] Amazon Web Services, "Listeners for your Application Load Balancers." *Elastic Load Balancing User Guide*. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-listeners.html Accessed November 3, 2026.

[5] Amazon Web Services, "Target groups for your Application Load Balancers." *Elastic Load Balancing User Guide*. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html Accessed November 3, 2026.

[6] Amazon Web Services, "Health checks for Application Load Balancer target groups." *Elastic Load Balancing User Guide*. https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html Accessed November 3, 2026.

[7] Amazon Web Services, "Control traffic to your AWS resources using security groups." *Amazon VPC User Guide*. https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html Accessed November 3, 2026.

[8] Amazon Web Services, "Subnets for your VPC." *Amazon VPC User Guide*. https://docs.aws.amazon.com/vpc/latest/userguide/configure-subnets.html Accessed November 3, 2026.